

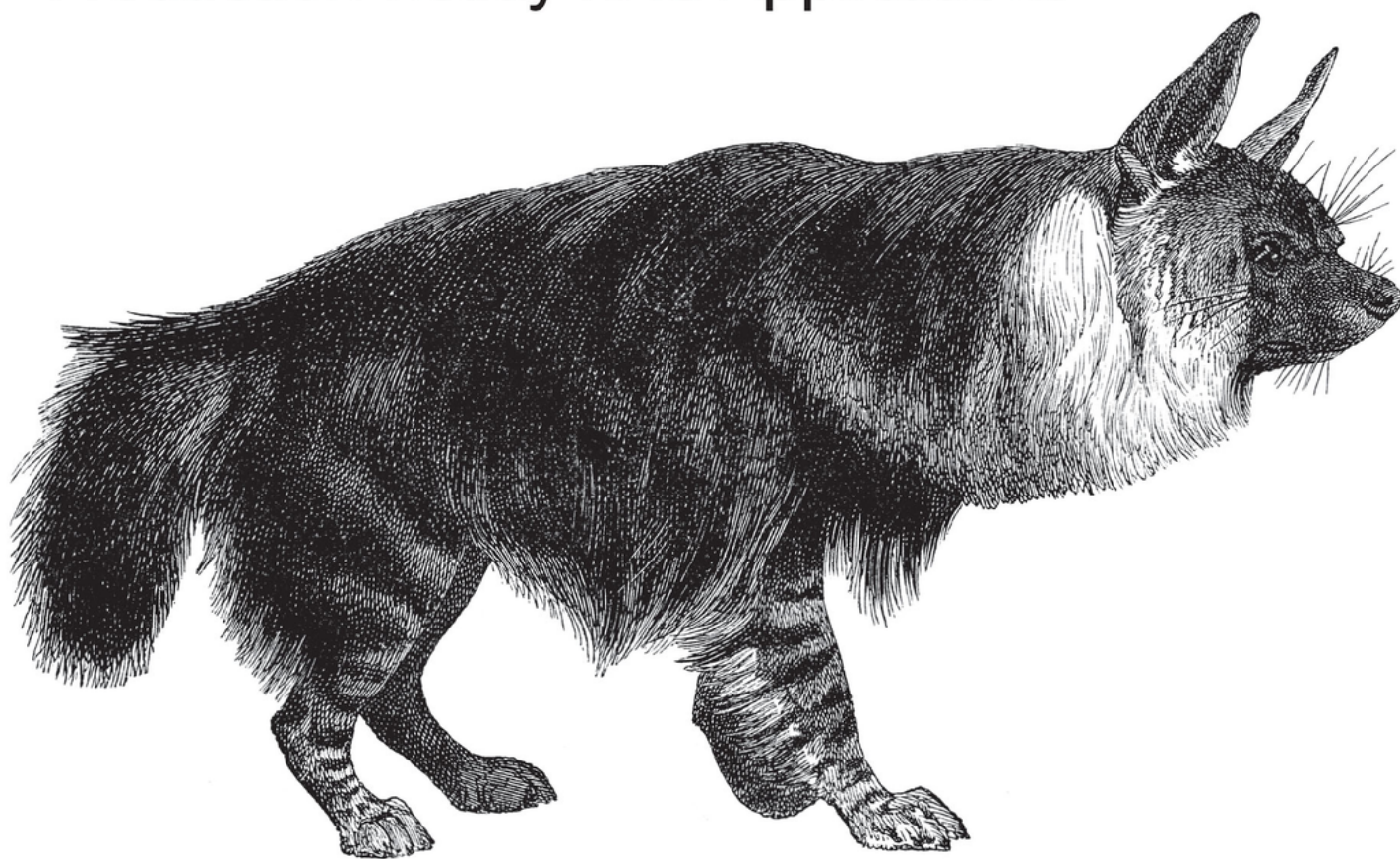
O'REILLY®

Early
Release

RAW &
UNEDITED

Hands-On RAG for Production

Design, Develop, and Deploy
Production-Ready RAG Applications



Ofer Mendelevitch & Forrest Bao

Hands-On RAG for Production

With Early Release ebooks, you get books in their earliest form—the authors’ raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

Ofer Mendelevitch and Forrest Bao

O'REILLY®

OceanofPDF.com

Hands-On RAG for Production

by Ofer Mendelevitch and Forrest Bao

Copyright © 2025 Ofer Mendelevitch and Forrest Bao. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North,
Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

- Editors: Nicole Butterfield
- Development Editor: Melissa Potter
- Production Editor: Jonathon Owen
- Interior Designer: David Futato
- Cover Designer: Karen Montgomery
- Illustrator: Kate Dullea

- June 2026: First Edition

Revision History for the Early Release

- 2025-05-07: First Release

See <http://oreilly.com/catalog/errata.csp?isbn=9798341621718> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. Hands-On RAG for Production, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors and do not represent the publisher's views. While the publisher and the authors have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

979-8-341-62166-4

[FILL IN]

OceanofPDF.com

Brief Table of Contents (*Not Yet Final*)

Chapter 1: Introduction to Retrieval-Augmented Generation (RAG)
(available)

Chapter 2: The Base RAG Stack (unavailable)

Chapter 3: Scaling Your RAG Stack (unavailable)

Chapter 4: Deploying RAG to Production (available)

Chapter 5: The Turnkey RAG Stack (unavailable)

Chapter 6: Evaluating Your RAG Application (unavailable)

Chapter 7: Agentic RAG (unavailable)

Chapter 8: Multimodal RAG (unavailable)

Chapter 9: Knowledge-Enhanced RAG (Graph RAG) (unavailable)

Chapter 10: Future Directions and Conclusion (unavailable)

Chapter 1. Introduction to Retrieval Augmented Generation (RAG)

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 1st chapter of the final book.

If you’d like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at mpotter@oreilly.com.

Large language models (or LLMs) like Anthropic’s Claude, OpenAI’s ChatGPT, or Google’s Gemini are extremely powerful at generative tasks like answering questions, writing code, or processing data.

This is indeed a brave new world, powered by AI.

LLMs are trained on massive collections of text and code, encompassing diverse sources like books, articles, code, and web pages. Their impressive capabilities notwithstanding, there is one problem. LLMs are bound by the knowledge exposed to them during training, and hence, they cannot answer

questions, write code, or provide any other service that is grounded in private datasets within a company.

In other words, if you ask ChatGPT a question, where the answer requires access to an internal database, documents on a private Google Drive, or information written in Confluence or Notion, you are not likely to get the right answer. Instead, LLMs will still provide some answer that will likely be hallucinated (when the LLM generates content that is unsupported by either world knowledge or the information fed to its prompt, we call that a “hallucination”) or completely incorrect, because they lack knowledge that resides in private data, or data that is too new or too niche (e.g., a not-yet-popular Python library) to be included in their training data.

Not only is training LLMs using a tremendous amount of data costly and tedious, it is also unmanageable to collect all the documents in the world, popular or niche, public or private, in real-time. An LLM will always be partially outdated and won’t cover the entirety of human knowledge.

A popular solution is called *Retrieval Augmented Generation*, or RAG, which instantly and efficiently boosts LLMs with knowledge from any source of documents, making it a popular choice for building generative AI applications grounded in domain-specific or private data.

RAG works by adding real-time retrieval to generative AI, allowing an LLM to access relevant facts from massive amounts of data outside of the

model's training set. By dynamically combining retrieved information with generative capabilities, RAG represents a significant leap in creating AI systems that are reliable, more flexible and that provide more accurate responses with significantly reduced levels of hallucination.

How Does RAG Work?

As shown in [Figure 1-1](#), RAG has two steps, the R (retrieval) step and the G (generation) step. When a user issues a *query* to a RAG system, the “R” step kicks in first to retrieve information that is most relevant to the question (or query). Then follows the “G” step, where a response is generated by tasking a large language model to analyze the retrieved information and the query, and craft a proper response to the query grounded in the facts retrieved.

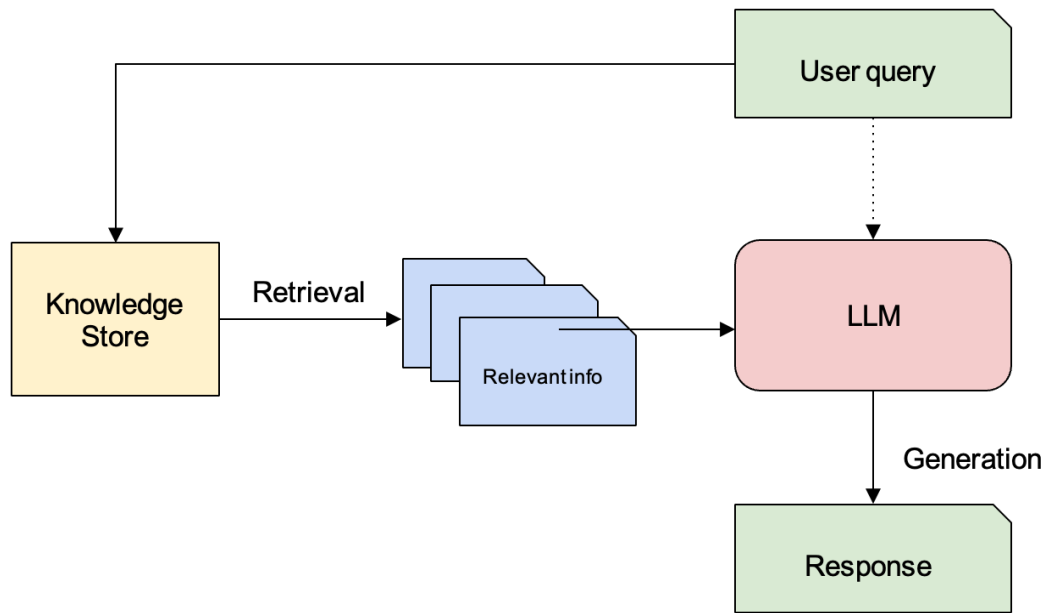


Figure 1-1. Basic RAG architecture

Let's look at an example. Suppose we are using RAG to build a chatbot that answers medical questions, and is grounded in medical books, papers, and patents.

For the query “What are the effective treatments for diabetes?”, the R step will, at least ideally, bring back information that is related to the treatment of diabetes and leave treatment of other conditions or causes of the condition out. Then, the G step will separate, in the hidden state of an LLM, effective and ineffective treatments that have been tried and only present effective ones to you.

The word “augmented” in “Retrieval Augmented Generation” suggests that the retrieved information is added (that's the meaning of the word

“augment”) to the prompt of an LLM for generation. A RAG prompt typically looks something like this:

```
"""
```

```
Here is a user query: {query}.
```

```
And relevant context:
```

```
{context}
```

```
Please respond to the user query using the context.
```

```
"""
```

Thus, we task the LLM with a question-answering task: look at the source facts (context) and respond to the query using information and facts provided in the context.

In many ways, the difference between pure LLM use and RAG is similar to the difference between a closed-book test and an open-book test. In a closed-book test, students must rely solely on their memory and understanding. No textbooks, notes, or other reference materials are allowed. Similarly, pure LLM usage means that all the information you get is based solely on the dataset included during the LLM training. Such knowledge is stored in the parameters of an LLM, which is an artificial neural network whose behavior is determined by the values of its weights, and thus referred to as the *parametric knowledge*.

In contrast, in an open-book test, students can consult textbooks, notes, or other approved materials during the exam. This setup allows them to refer back to detailed information if needed, and is exactly how RAG works - the retrieval step provides additional information to the LLM in real time.

With that knowledge in place, let's now go a level deeper to understand what components are required for RAG and how the ingest and query flows work.

The Blueprint of a RAG Stack

Let's look at how RAG works in more detail, and specifically the various components in the RAG stack. [Figure 1-2](#) below depicts two flows: the ingest flow and the query flow.

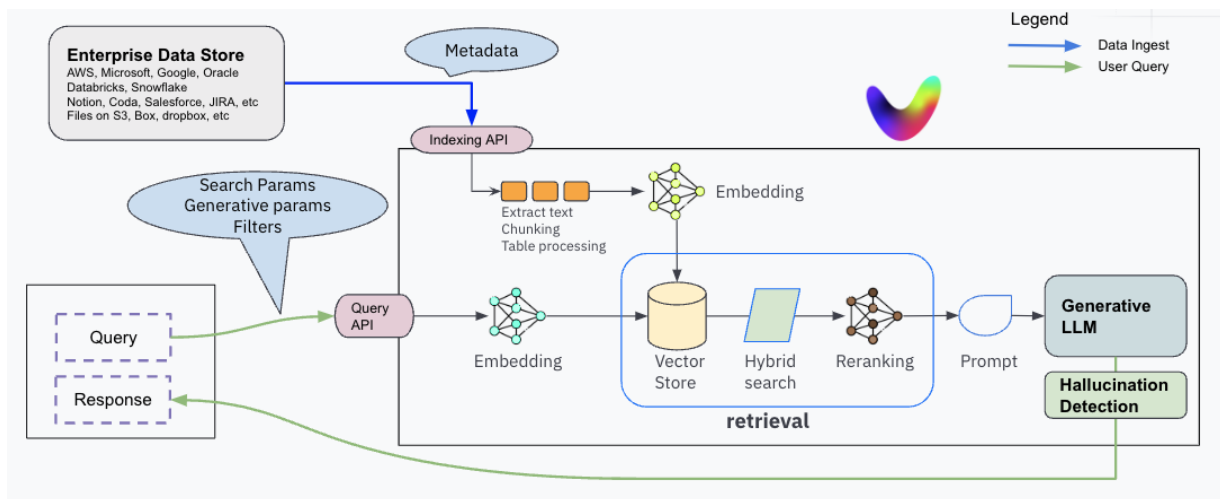


Figure 1-2. The RAG stack

The ingest flow performs those functions needed to extract the data from its source (like a database, a set of PDF files on S3, text on Notion, etc) and index it into the RAG stack.

The query flow performs the full processing of a user query - retrieves the right facts and uses the LLM for generative summary, resulting in a response to the end user.

Let's look at these in more detail.

The Ingest Flow

During data ingestion, the RAG system first converts the input data (against which user queries will be answered) into vectors, also known as embeddings (or vector embeddings). These vectors represent the semantic meaning of the text, and as we will see Chapter 2 are a very powerful mechanism to match a query to a fact. The vector embeddings are then stored in a special database called a *vector database* or *vector DB*.

Alongside each vector, the actual text is also stored as it is needed for query time processing. The step of converting data into vectors is often referred to as *indexing* or *embedding*.

Note that in the RAG world, the word index, dataset or corpus are somewhat synonymous - they all ultimately mean the storage mechanism where the text data is stored. With more advanced RAG, the index may also contain tables, charts, images or videos.

The Query Flow

The query flow starts with converting the user query into an embedding, and then the vector DB performs a similarity match operation between the query embedding and all possible matching text (the facts) in the vector DB. Ideally, retrieved pieces of text contain facts that are highly relevant for answering the user query.

Looking up information using embeddings is called *semantic search*. By applying similarity search in the embedding vector space (that humans are usually unable to understand) it can match queries with relevant text answers. In chapter 3 we will discuss advanced approaches to retrieval including hybrid search (combining vector search with the more traditional keyword search), as well as reranking.

Once we have the relevant facts, the generation step works as follows: the RAG query flow then crafts a dedicated prompt template, like what we have seen above in “How Does RAG Work?”, to instruct the LLM how to produce a response that answers the user’s query using information in the retrieved results.

Importantly, good RAG pipelines often instruct the LLM to produce references or citations, so that the response includes not only the raw text of the answer, but also points to the source of the knowledge that the response is grounded upon.

We're not done yet!

After the LLM sends back its response, a typical RAG query flow applies guardrails to make sure the response meets expected quality. First and foremost is hallucination detection — namely, validating that the LLM's indeed used the facts provided to it to create a response that is factually consistent with the facts. In other words, we're trying to check that the LLM didn't make things up. Additional types of guardrails include detection of bias, toxic or harmful responses, or otherwise disallowed content, as we'll see later on in chapter 3.

So that's how the RAG ingest and query flows work. But you may be wondering if RAG is the only approach to use LLMs with your own data. Let's take a look at a few alternative approaches and compare them to RAG.

RAG vs. Other Approaches

When first entering the world of LLMs and RAG, there are quite a few approaches that look similar to RAG, at least in function, but often have significant downsides or are just too simplistic to support real-world, production-scale use-cases. Let's discuss those in a bit more detail.

RAG vs. “chatting with PDF”

You may find that RAG looks similar to “Chat with PDF” - a category of applications that answer user queries based on a set of documents.

Although it’s certainly possible to implement a “Chat with PDF” application using RAG, most “Chat with PDF” applications use the following simple (although non-scalable) approach: they put the full text of the PDF into the LLM prompt, followed by the questions. This approach works for small PDFs, as modern LLMs now support a context length of 128K or even 1M tokens. In fact, you might be able to fit 10s or 100s of PDFs into such a large context window.

However, this clearly does not scale to enterprise applications, where often there are hundreds of thousands of documents available and even very large context windows sizes won’t be enough.

There are a few other limitations worth considering.

First is cost: LLMs are expensive to run. By feeding all documents into the LLM, you also feed information that is irrelevant to the query to the LLM, which is a waste. Instead, RAG selectively feeds relevant information to the LLM, making it cheaper, faster and scalable to any size.

Second is latency: even LLMs that can process long sequence lengths may take a while to process, resulting in high latencies and a frustrating user

experience.

Third is accuracy: imagine an enterprise application where we just want to get an answer to the question, from documents across Google Drive, Notion, Sharepoint and a set of PDFs on S3. With “chat with PDF” someone still has to identify which documents are relevant and feed those into the LLM as mentioned above. Now we’re back to retrieval and it starts looking exactly like RAG.

RAG vs. fine-tuning

Developers building Generative AI that utilize LLM with proprietary data often consider using *fine-tuning*. This involves taking a pre-trained model and further training it on specific, domain-relevant data for a few more epochs.

This approach allows the model, at least in theory, to internalize nuances, terminology, and patterns unique to your data, effectively embedding the knowledge directly into the model’s parameters.

So what’s wrong with fine-tuning?

First and foremost, fine-tuning is a difficult task, which requires careful preparation of the data and deep expertise in deep learning to avoid issues like overfitting, forgetting general language competencies, or introducing biases presented in the training data. Even if you have a team with that level

of expertise in deep learning training of LLM, your data may just not be large enough or clean enough for effective fine-tuning.

Not only that, fine-tuning tends to be quite expensive (in terms of GPU cost), and you must ask yourself: how often do I need to fine-tune? If your dataset is static, then it's not much of an issue—you fine-tune once and you are good to go. But in most real-world enterprise use cases, data often gets updated frequently—would you fine-tune every day? Once a week? That is unlikely to be a cost-effective solution.

Another often overlooked, but quite important advantage of the RAG approach is that of access controls. Imagine you have a dataset that includes documents from multiple departments: engineering, HR, finance and legal. If you try to use fine-tuning, you effectively incorporate all this data into the model weights.

We like to think of this as the Borg effect (“We are the Borg. Resistance is futile”) - just like the Borg in StarTrek assimilates or integrates beings, cultures, and technology into the Collective, fine-tuning integrates all the knowledge it trains on into the model weights.

Now what if an employee of the company asks a question and the fine-tuned LLM responds based on confidential information that should only be available to the CEO or the HR department? With fine-tuning, the

information is one single “blob” and you cannot separate documents visible to the CEO from those that are globally visible to all employees.

Of course, you might consider finetuning different LLMs using data accessible for each user group or department. But that results in multiple LLMs being fine-tuned—each needs to be hosted separately and you would need to build a way to route queries to the right model. Overall this approach is not very scalable and quickly adds to cost and complexity.

With RAG, you can easily implement access controls within the query retrieval step - by adding permission-based metadata fields in the datastore and using filtering at query time, and in this way ensure compliance with company policies (we will discuss this in more detail in chapter 3).

It is important to note that there is nothing that prevents you from using a fine-tuned LLM as part of your RAG stack. If you have a lot of internal data, and the expertise to properly fine-tune a proprietary LLM that is fine-tuned on your data, you can use that in the generative step of RAG.

To summarize, chat-with-your-PDF and fine-tuning are valid approaches in some cases, but have significant limitations when it comes to enterprise deployments. Now let’s explore the key benefits of RAG and what makes it a great approach for mission-critical and large-scale enterprise applications.

Key Benefits of RAG

So now that you understand how RAG compares to other potential approaches, let's discuss the main benefits of RAG itself in more detail:

RAG is Scalable and Efficient

RAG is an efficient approach for grounding generative AI applications in private datasets that easily scales to hundreds of thousands, millions or even more documents.

The retrieval engine at the core of the RAG makes this possible. Search is a hard problem that has been researched for decades, providing ample approaches that can be used in RAG. And we know that search (and thus RAG) scales linearly with the number of documents, whereas using an LLM directly only scales quadratically, due to its use of the (now famous) [self-attention mechanism](#).

RAG helps reduce hallucinations

We use the word “hallucination” to describe the scenario when an LLM generates content that is unsupported by either world knowledge or the information fed to its prompt.

Due to its design, RAG helps reduce hallucinations as compared to asking an LLM in a closed-book fashion. The reason is this: because

we provide LLM with a set of facts, retrieved from the source dataset, that are relevant for the user query, it will (if built properly) use those facts to provide a good answer, based on these facts.

If it does not have relevant facts, RAG will just respond with “I don’t know” because that is how we instruct it to behave. In contrast, an LLM will always provide some response based on its training set and if it does not have that information it will in many cases make something up.

RAG Improves Explainability

RAG uses retrieved information to answer user queries, so it is a common practice to implement citations (like “[3,5]”) at the end of each sentence generated by the RAG pipeline, if requested to do so.

LLMs in a RAG application can be further instructed, through proper prompts, to explain how it reaches an answer by processing the retrieved information and reasoning about it. Such high explainability is unmatched when an LLM only uses its parametric knowledge (extracted from training set) to answer questions because it is almost impossible to reconstruct the source from neural network weights.

Instant addition and removal of knowledge

The response generated by an LLM in RAG depends on the retrieved data that is fed into the LLM (assuming that data is relevant of course). This means the knowledge accessible to the LLM can be instantly added or removed.

The LLM will have no memory of the knowledge given to it. It only needs the right facts to be retrieved during query time.

Compare that to using a frontier model or a fine-tuned model, where with any new data item, retraining is required, and it is almost impossible for the LLM to forget a specific piece of knowledge, due to the complex and non-transparent nature of neural networks.

Access controls and security

Just like adding or removing knowledge can be done by enabling or masking out data accessible in a RAG pipeline, we can implement access controls in a similar fashion. By adding permission information to documents during ingestion (e.g. as metadata), we can direct the query flow to include or exclude certain documents based on their permissions.

Properly supporting access controls is often a critical requirement in an enterprise application of RAG, preventing leakage of data that a user is not authorized to see into the RAG responses.

As we can see, RAG offers capabilities that are very attractive for enterprise applications, especially for organizations that are required to provide strict access controls, strong security, and most importantly care about response quality and reduced hallucinations.

Let's look at some of the specific use-cases that these organizations use RAG for.

RAG Use Cases

This ability to utilize the power of LLMs, while augmenting them with private data makes RAG applicable to nearly any application where an LLM will be used inside an enterprise, because most enterprise applications require access to their own private data.

That is not to say that ChatGPT, Claude or Gemini are not useful as stand-alone tools for employees. They are. They can be used very effectively to improve one's productivity: for coding, marketing, or other tasks that require general world knowledge and many other uses.

But for any applications where the LLM needs access to internal data, RAG is best.

Let's review some common Enterprise RAG use cases.

Virtual Assistants and AI Chatbots

Virtual assistants and chatbots can serve as the first line of customer interaction. This is valuable both as an externally facing chatbot interacting directly with consumers or as an internal tool for customer service agents.

For example, in an airline - customer support agents use a virtual assistant to help them with their daily tasks, providing answers to common questions they may face when speaking to customers on the phone. A different chatbot can be deployed externally, directly serving the airline customers with any question they have.

In this use case, it is common to point your RAG application at relevant internal knowledge bases, such as previous customer support logs, airline FAQ or website information, as well as other internal documents around policies.

Deploying virtual assistants in this manner often shows a positive impact on customer service metrics, helping to dramatically reduce response times, reduce the overall volume of support tickets, and increase first-contact resolution rates. The technology ensures that every interaction is informed by the most current and comprehensive data, thereby elevating the overall customer experience.

The number of applications that Chatbots and virtual assistants can serve is quite large. As long as you have an appropriate dataset that encapsulates the

knowledge you want to ground the assistant on, you can simply point your RAG application to that dataset and deploy a virtual assistant.

As another example of this common use-case, let's look at education. Universities and schools can deploy a chatbot to help answer student questions, since it is nearly impossible for every student to have access to a teacher or tutor at any time. Using an AI assistant built with RAG, we can provide every student a teacher for any subject, any time and anywhere. Just hook a RAG system with course materials authorized to or created by the teacher such as textbooks or notes, then RAG will be able to answer a student's questions in the scope of such textbooks.

Figure 1-3 shows a RAG-powered intelligent tutoring and adaptive learning platform. First (Triangle 1), course materials are fed into the ingestion server, which processes them and stores them into the vector DB. Then the agent can interact with a student through chatting as both a tutor (Triangle 2) and an examiner (Triangle 3). In the tutor mode, The student asks questions and the agent replies with answers. For every question the student asks, the agent mobilizes its LLM and query the vector DB to generate the answer to the students. In the examiner mode, it proactively generates questions to the student and grades the student's responses. It can even re-generate new questions based on low-grade problems to reinforce the student's understanding of knowledge.

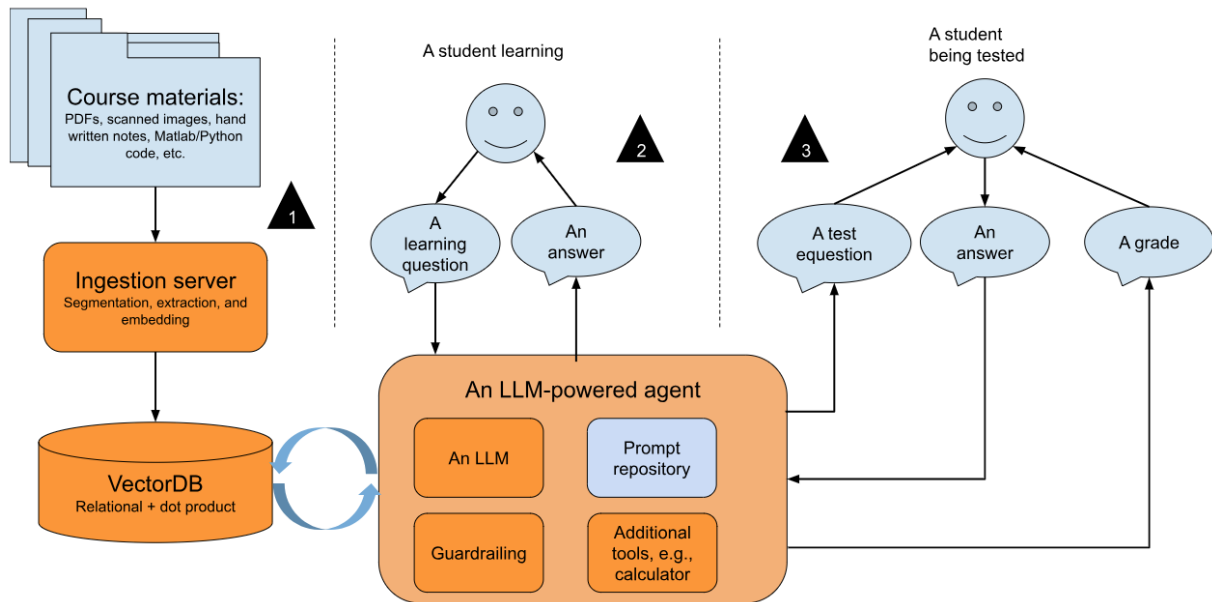


Figure 1-3. A RAG-powered intelligent tutoring and adaptive learning platform.

Enterprise Knowledge Management & Internal Search

In an enterprise setting, employees often face the challenge of finding the right information amid vast and diverse data sources, especially as data is often stored in multiple systems: as files on Google Drive, Notion, Salesforce, Hubspot, JIRA, Confluence and any other system.

RAG modernizes enterprise search by combining the strength of retrieval, which was common in traditional enterprise search systems, with an LLM that adds the generation and information processing/reasoning capabilities. By ingesting all relevant enterprise data sources into your RAG application, when an employee submits a query—be it for policy details, historical meeting documents, or asking for technical specifications—the system

extracts the most relevant content, and then generates a clear, summarized response.

This process replaces the traditional, often time-consuming process of looking at top 10 results of a search, and reading each of those documents, while trying to form a coherent and accurate response in your head.

The benefits for companies using this approach are manifold. Employees save time that would otherwise be spent sifting through numerous documents, and avoid missing critical information. This efficiency gain boosts overall productivity and allows teams to focus on higher-value tasks rather than administrative searches.

By continuously keeping your data sources refreshed and up-to-date, RAG-based knowledge management systems keep pace with the rapid changes within an organization. This dynamic adaptability provides a marked improvement over static, legacy search tools that often become outdated quickly.

Automated Content Creation & Document Summarization

Content creation in enterprises is quite common, including tasks like generating internal reports or creating marketing articles or blog posts. These types of tasks often require meticulous research and fact-checking.

RAG offers a powerful solution by automating the creation process. When tasked with generating content, the RAG system retrieves the latest, relevant data from multiple sources and uses it to produce well-structured drafts or summaries, which can then be reviewed if needed by a human, as a final review step. This can dramatically reduce the amount of time and effort required with manual research, and often results in more accurate content.

The positive impact extends to brand reputation as well. With content that is accurate and promptly generated, companies can maintain a consistent and authoritative voice across all channels. This level of responsiveness and reliability can be a major competitive advantage over legacy processes that may take longer (competing with other priorities) and result in less accurate artifacts.

Generating Attractive and Effective Personalized Ads

Advertisements need to be attractive and effective. Conventionally, the same ad is delivered to all its target audiences without factoring in what the user is doing or talking about online. With RAG, we can generate ads with more up-to-date, and personalized information to produce ads that differ from person to person and are potentially more effective.

Figure 1-4 depicts a RAG-powered system to generate compelling ads on-the-fly based on user context, e.g., what the user was/is talking about, watching or browsing. For example, if a user is chatting about running, products related to running or sports will be pulled from the pre-ingested vector DB, and a personalized ad will be generated using both the product information and the context about the user (e.g. their previous purchases).

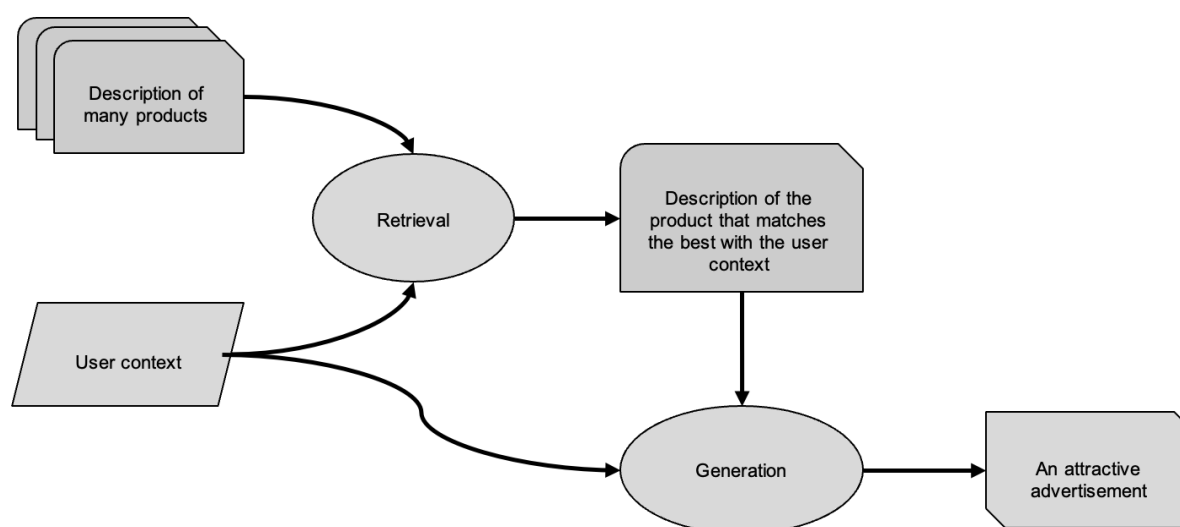


Figure 1-4. A RAG-powered pipeline for generating personalized ads on-the-fly.

The advantage of RAG-powered ad generation is clear: we can use the powerful semantic search in RAG for production recommendation, and not only show the products but create an ad for that product that matches what we know about that user.

As an example, let's say we want to advertise Acme Shoes, which are designed for both safety and hygiene. For a user who was recently talking about foot odor and is now talking about soccer, the ad can begin with

addressing the odor pain point with something like “Love soccer, but hate foot odor? Acme shoes are specially engineered to suppress microbes that cause odor”. To another user who was talking about safety, the ad can be “You don’t wanna give up safety to stay in shape. Acme shoes have reflective strips to protect you.”

Question Answering Systems

Question answering systems are designed to deliver precise answers to user queries by synthesizing information from diverse datasets, using RAG.

Unlike Chatbots or virtual assistants, which support multi-turn conversations, the form-factor here is that of a single question and single answer..

One common use case of question answering is for helping respond to Requests for Proposals (RFPs) or Requests for Information (RFIs). In the competitive sales landscape, speed and accuracy in responding to customer inquiries and proposal requests are critical. When a sales team needs to prepare a tailored proposal, the RAG system pulls relevant historical data, product specifications, pricing details, and customer interactions from internal databases, and then constructs a coherent, customized response.

The positive impact is clear and the benefits are considerable. Sales teams can produce high-quality proposals in a fraction of the time compared to

manual processes, thereby increasing their responsiveness and competitiveness. This automation minimizes the risk of human error and ensures that each proposal is backed by the latest and most accurate data (as opposed to copy-pasting from the previous proposal where data may not be up-to-date), leading to improved win rates and stronger customer relationships.

Medical & Healthcare Applications

In the healthcare sector, timely and accurate information can be a matter of life and death.

When a clinician needs to quickly review treatment guidelines or patient histories, a RAG application can retrieve relevant case studies, research articles, as well as the patient's medical record and all physician notes to generate a concise, evidence-based response.

What can be even more useful—that response can be tailored to each physician's specialty. For example, the summary might be different if you are a cardiovascular surgeon or a dermatology specialist—the information relevant to each is different.

By providing an accurate and contextualized medical summary, combining historical medical records with up-to-date medical information, we can help physicians be more effective in treating patients, reduce the likelihood of

missing critical information such as an allergy, and overall provide better treatment to their patients.

For healthcare providers, as well as insurance companies, the benefits are substantial - reducing the time needed to make informed decisions and thereby improving patient outcomes. They also help to reduce the cognitive load on clinicians by presenting synthesized, easily digestible information instead of overwhelming raw data.

And of course, patients benefit from more precise and personalized care. With faster access to critical insights and reduced risk of outdated or incorrect information, medical practitioners can deliver treatments that are both timely and effective.

Legal & Compliance Research

Many regulated industries like healthcare and financial services are required to comply with various laws and regulations. Understanding the full complexity of each legal requirement and regulation, and how it applies to your business, is often complex and requires legal research where precision and reliability are paramount, as errors can lead to significant financial or reputational damage.

A RAG-based system can assist legal and regulatory professionals by quickly retrieving relevant case law, statutes, and internal compliance

documents. This approach greatly improves upon traditional legal research methods, which often rely on manual searches through legal texts and databases, as well as internal data sources.

The benefits for legal professionals are immense. By automating a significant portion of the research process, RAG enables faster turnaround times on legal opinions, compliance reports, and case preparations. This efficiency not only reduces labor costs but also minimizes the risk of overlooking critical information.

In the last few sections we covered the basics of RAG, and its main use-cases. Later in the book we will also cover more advanced forms of RAG—the next section will give you a preview of some of these advanced techniques.

Advanced RAG

RAG was originally introduced in a [Facebook/Meta 2020 paper](#) at the 34th NeurIPS conference. Back then RAG was presented to only process textual data and there was only one round of information retrieval and LLM generation. Pretty simple.

Since then, RAG has progressed into a more advanced and powerful form. We will discuss some of these advanced RAG techniques in detail in later

chapters, but here we provide a quick overview of some of these key techniques.

Agentic RAG

Agentic RAG is an evolution of RAG_instead of a one-shot process to retrieve relevant information and generate a response, Agentic RAG incorporates autonomous AI agents into the pipeline. These agents add capabilities such as:

Iterative and Multi-Step Retrieval

Instead of fetching context only once, agents can re-retrieve and refine the information if the initial data isn't sufficient.

Dynamic Tool Integration

Agentic RAG can leverage multiple external tools (like web search or API calls) to access varied sources of knowledge, rather than relying solely on pre-ingested knowledge.

Advanced Reasoning and Adaptability

The agents can decompose complex queries, plan retrieval strategies, validate information, and even coordinate among specialized sub-agents to handle multi-part tasks.

In summary, Agentic RAG offers greater flexibility and robustness for handling complex, multi-faceted queries by dynamically orchestrating several retrieval and reasoning steps.

Multi-modal RAG

Initially, RAG was only used with textual information. Since then, RAG has been expanded to cover other modalities, such as tables, diagrams or charts. There are usually two approaches to incorporate these other modalities.

The first approach is to convert all information modalities into text (e.g., images to their captions), and then run the well-understood RAG pipeline in the text domain only.

The other is to leverage multimodal retrieval models and language models, such as visual language models (aka VLMs), or multimodal large language models (aka MLLM). In this approach, information in non-textual domains remain in their original modality. This information is then provided to the VLM during query time in the retrieval stage and used to generate the response together with textual data.

A more end-to-end approach on the rise recently is embedding the entire page and sending pages into an MLLM for generating the response. We will explain more in Chapter 8: Multimodal RAG.

GraphRAG

The RAG examples we have discussed so far use digitized information as is, without any human processing or extraction. Popularized by Microsoft, and quickly adopted by many of the graph database vendors, GraphRAG was proposed to address the limitations of conventional RAG (text only or multimodal) in “connecting dots”, with an approach that tries to improve the quality of retrieval by leveraging knowledge graphs.

As you saw earlier in this chapter, the goal of the retrieval step in RAG is simply to retrieve the most relevant facts required to answer the user query, and advanced RAG pipelines thus deploy not only similarity search, but also hybrid search or re-ranking.

Rather than relying solely on flat text embeddings, GraphRAG first processes unstructured documents to extract entities and relationships, and constructs a knowledge graph that captures the inherent connections within the data. This structured representation enables the system to support multi-hop reasoning (meaning connecting different pieces of information in the text to reach a conclusion that is not obvious or cannot be reached by reasoning once from one piece of information) and deeper context awareness when answering complex queries in the RAG context.

Conclusion

In this chapter, we introduce RAG, a common and effective approach to overcoming the inherent limitations of large language models (LLMs).

While these models excel in generating responses, writing code, and answering questions based on the extensive data they were trained on, they fall short when it comes to handling proprietary, up-to-date, or niche information that lies outside their training set.

RAG addresses this by integrating real-time data retrieval into the generative process, ensuring responses are grounded in relevant, external information.

We then introduced the architecture of a RAG system, outlining both the ingest and query flows, and the different steps in each flow.

We then reviewed alternatives to RAG (such as fine-tuning) and discussed the pros/cons of each approach, finishing up with a discussion of the benefits of RAG, the main use-cases for RAG in the enterprise, and some advanced techniques (which we'll discuss in much more detail later in this book).

Building a RAG application requires a lot of learning—from new types of systems components like vector databases to models like embedding, LLMs, and more. The rest of this book is dedicated to diving deep into each

of these components to gain a better understanding of how they work in practice and what is required to scale them to enterprise use-cases.

In the next chapter, you will learn how to build a basic RAG stack, and learn more about each component that is required: its function and how it works, and how it interoperates with other components.

[OceanofPDF.com](https://oceanofpdf.com)

Chapter 2. Deploying RAG to Production

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 4th chapter of the final book.

If you’d like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at mpotter@oreilly.com.

Now that you know all the components of a RAG pipeline, both basic and advanced, you can easily put together a pretty good proof-of-concept (POC).

For a first POC, it is typical to pick a use-case with significant value to the organization, where the initial investment is relatively low. This way you get to learn how this actually works, and understand first-hand how RAG works.

Getting a RAG proof-of-concept up and running is a lot of fun. You take a super powerful Large Language model, point it at your documents or data, use cosine similarity with a vector database, and voila—you can start asking questions and get real answers, based on the content of the documents.

If you do this as a side-project, it takes only a modest amount of time and effort. However, if your goal is to build a production grade RAG application that is scalable, secure, fast and that provides a mission-critical service to your company—that's a whole other story.

Moving from POC to a production-grade deployment of a RAG application presents enterprises with many challenges spanning technical, operational, and organizational domains. As you scale your RAG application, you often confront latency bottlenecks, vendor integration complexities, data security requirements, and interdisciplinary expertise gaps.

In this chapter we dig deeper into some of these challenges, and where possible - discuss strategies to address them or minimize any negative impact.

Challenges with RAG in Production

A scalable, production grade RAG stack is much more difficult than it first appears to be. There are many hurdles including response quality, latency,

security, support and cost.

Response Quality and Reduced Hallucinations

Whether you are using RAG to build an AI assistant, a question answering application, for automated RFP (request for proposal) responses, or any other use-case, the quality of the response from your RAG pipeline is often the most important feature to focus on.

Users tend to disengage from an application they cannot trust, so if many of the responses are inaccurate or include hallucinations, user trust in this application dramatically reduces, rendering it essentially unusable.

It's important to understand the various reasons that may result in low quality responses, so that you can identify the cause and address it.

Reason 1: No Relevant Data

Imagine a RAG pipeline that is grounded in information from user manuals about Samsung TVs.

If the user asks a question about a specific Samsung TV model, but the user manual for that model is not included in the data, then clearly the system has no information to ground its response in.

Quite often, what might occur is that retrieved facts will not be relevant, and the LLM will use those facts to generate a response, which might

clearly be incorrect.

By tracking user queries and response quality, you can identify this kind of issue, and update your RAG dataset to include all the necessary information to respond accurately to any user query.

Reason 2: Weak Retrieval Pipeline

Assuming we do have the right information in the dataset, the next culprit is often the quality of your retrieval pipeline. Most POCs start with a simple vector search (aka “semantic search”) approach, using a vector database.

As you scale to production, the number of documents that are available grows, making the task of accurate retrieval much more difficult since there are a lot more potential matches for any given query, requiring more sophisticated filtering and ranking mechanisms. Furthermore, as the dataset grows in size, the indexing used by vector and keyword search mechanisms become larger and more complex, demanding efficient algorithms to update and search them quickly.

Often, you need additional capabilities such as hybrid search, or various types of re-rankers (as discussed in Chapter 3) to achieve a high quality retrieval pipeline. This often translates into distributed storage and requires mechanisms to ensure consistency, fault tolerance, and efficient data retrieval, all of which add layers of complexity.

The bottom line is this: RAG is “garbage-in-garbage-out”: if you don’t invest enough in a strong retrieval pipeline as you scale to production, the facts provided to the LLM will not be as accurate as in your POC, and the quality of responses will degrade.

Reason 3: LLM hallucinations

Even with perfect retrieval, LLMs often struggle to faithfully incorporate the provided evidence or facts in the source documents (or chunks) into the final response, resulting in what is known as “hallucinations,” which we briefly discussed in [Chapter 1](#).

One complicating factor is the variability and potential incompleteness of the retrieved facts. In many cases, the documents returned by the retrieval component may not cover the full scope of information required to answer the query, leading the generative model to fill in gaps with inferred information. This gap-filling behavior can inadvertently result in hallucinations, and detecting these inaccuracies is further complicated by the fact that the generated text may be partially supported by the retrieved data, creating a “spectrum of factuality” rather than a clear binary between true and false.

As you consider your production RAG application, you need to choose an LLM that has a low rate of hallucinations, as well as consider implementing

advanced techniques to detect and correct hallucinations in your RAG pipeline (as we discussed in Chapter 3).

This adds significant additional research and development effort, well beyond what you've done in the POC.

Reason 4: Prompt Design

The basic prompt for RAG can appear quite simple, as you saw in [Chapter 1](#). But engineering a better prompt for your RAG system can have a significant positive impact on quality of responses.

For example, your basic prompt may be:

```
prompt = """
Use the following pieces of context to answer the
{context}
Question: {question}
Helpful Answer: """
```

A common improved prompt may look like this:

```
prompt = """
Use the following pieces of context to answer the
{context}
```

```
Question: {question}  
Helpful Answer: ""
```

Providing specific instructions to the LLM, such as shown here in “If you don’t know the answer, just say that you don’t know, don’t try to make up an answer”, can be a powerful way to improve response quality, especially as your production RAG covers a lot more documents and edge-cases that a more basic prompt may not cover.

Careful prompt design is thus crucial for curbing low quality responses. It also requires significant testing across a multitude of queries.

High Latency

Enterprise RAG systems must reconcile the computational load of semantic search, hybrid search, reranking and any other component in your RAG pipeline with user expectations for a quick response time.

During prototyping it’s common to prioritize functionality over speed, and as you move to production deployment, the application needs to adhere to more stringent latency thresholds comparable to those of the publicly available ChatGPT, often in the range of a few seconds. In addition, the amount of data during POC is usually a small fraction of the size of data in production, and that growth in scale can easily result in much higher latency.

As your RAG pipeline scales, increasing data volumes, more complex retrieval techniques, and the integration of advanced large language models may all contribute to undesired high latency. It's essential not only to keep the average latency within acceptable bounds but also to control tail latencies (e.g., the 95th percentile) that can degrade the overall user experience, especially under complex or resource-intensive queries.

In order to mitigate high latency, you will need to consider a variety of techniques including parallelization, using alternative models, software or hardware acceleration, auto-scaling, more efficient data indexing, caching, and adaptive query processing, to name a few.

Continuous monitoring helps identify bottlenecks both in real time (where auto-scaling can help) as well as more systemic latency issues, and correct them. This is especially important as you continue to improve your RAG stack, implement advanced retrieval techniques, advanced indexing or advanced techniques like Graph RAG.

Data Security and Privacy

Production RAG deployments must implement defense-in-depth strategies across three critical attack surfaces: ingestion layer, vector database and the generation step. Let's dig into each of these a bit more.

Ingestion Layer Security

Like any ETL (Extract, Transform, Load) pipeline, your ingest flow needs to use standard encryption protocols to ensure safety during data movement from the data sources to your RAG pipeline. Furthermore, your RAG implementation needs to ensure the same security protocols are implemented throughout your RAG pipeline and in every component—document extraction, chunking, embedding, and storage in the vector database.

If your data includes Personal Identifiable Information (PII) or Protected Health Information (PHI), you need to consider your redaction strategy, while making sure redaction won't result in reduced response quality due to loss of information. If you need to comply with the [ISO 27001](#) standards for information provenance, hash-based data lineage tracking becomes essential.

Vector Store Safeguards

In a RAG pipeline, the vector DB serves as the repository for both the vector embeddings and their associated textual data. With hybrid search, a separate text database that is optimized for fast keyword-based retrieval is also included in the implementation.

In both cases, the core requirement here is both encryption (in place and in transit) as well as role-based access controls—to comply with your

company's security policy or to enforce GDPR's "minimum necessary" principle (storing only the essential data required for your application's functionality, regularly reviewing and purging unnecessary data, and implementing privacy-by-design principles to minimize exposure of sensitive information).

As with any secure enterprise system, our vector and text databases need to implement network security best practices as well as continuous monitoring and incident response protocols.

Preventing Data Leaks

Your RAG datastore includes all the data that should be available to drive RAG queries. It includes a diverse array of documents and data, each subject to different permission levels within your organization. For example, some documents may be accessible to all employees, while others remain confidential and are only visible to senior management, such as the CEO or to the HR department.

Consequently, it is essential to integrate a robust filtering mechanism into the query flow, to avoid potential data leaks, as we've seen in Chapter 3.

This filter leverages the company's RBAC (role-based access control) policies to ensure that only data authorized for the querying user is passed to the LLM. By doing so, you prevent unauthorized access and mitigate the risk of exposing confidential information.

This is why regular audits of the query flow, thorough testing of filtering mechanisms, and continuous monitoring of external interactions are critical measures to detect and prevent such incidents.

In addition to internal access controls, a common privacy concern is data leakage to LLM providers. Specifically when interacting with LLMs hosted by outside vendors (such as OpenAI, Anthropic or Google), the call to the LLM results in sending the internal data over the network to an externally hosted LLM to produce the generative response. This introduces risk that sensitive data may inadvertently be stored in a way that you did not intend. External LLM providers might log these queries or retain temporary caches of the input data, potentially leading to data leakage. Such exposure could occur even if the data is anonymized, as patterns or metadata might still reveal sensitive insights.

This often leads enterprise RAG applications with highly sensitive data to consider on-premise deployment models, as well as using open-source LLMs such as Llama2 or DeepSeek, which can be hosted within your data center or VPC without any potential risk to your data.

LLM Generation Guardrails

The LLM is responsible for producing the final output by combining retrieved data with the user's query. To align with company policies and

regulatory requirements, your RAG application must incorporate robust protections that prevent the generation of disallowed content.

These safeguards (often called guardrails, which we introduced in [Chapter 1](#)) should address issues such as hate speech, biased language, or any other forms of harmful or inappropriate output. Implementing these controls might involve:

Content Filtering

Integrating real-time filters that screen the LLM's output for disallowed content before it reaches the end user.

Compliance Audits

Periodically reviewing the generated responses against updated internal and external guidelines to ensure ongoing adherence to policy standards.

Furthermore, in enterprise production environments, you need to mitigate the risk of *prompt injection attacks*, an emerging security risk where adversaries craft input queries designed to manipulate or hijack the behavior of the LLM. Here are common strategies to consider:

Careful Prompt Design

Develop prompts that clearly define the task and constrain the context in which the LLM operates. This helps ensure that the

model's response is directed and less susceptible to manipulation.

Input Sanitization

Rigorously sanitize and validate user queries before they are passed to the generation stage. This process involves detecting and neutralizing malicious or unintended inputs that could lead to prompt injection.

Monitoring, Tracing, and Logging

Establish comprehensive logging and monitoring systems to track the performance of the RAG pipeline, identify potential prompt injection related vulnerabilities, and facilitate rapid response in the event of an incident. Optimizing a RAG pipeline often requires tracing to deeply inspect how each pipeline execution performs.

User Feedback Integration

Incorporate feedback loops that allow end users to report problematic outputs, thereby enabling continuous refinement of the system's safety and compliance measures.

By integrating these detailed strategies into your RAG application, you not only enhance the quality and safety of the generated responses but also reinforce the overall integrity and trustworthiness of your system in enterprise settings.

Vendor Chaos and Integration Woes

Building a production-grade RAG stack involves more than just assembling a vector database, an embedding model, and a generative LLM. As your stack evolves, you may need to integrate additional components to maintain high-quality responses and robust performance.

These components may include:

Content Extraction

External APIs for extracting text from PDFs, Word documents, or PowerPoint files. If you implement advanced chunking approaches or even GraphRAG, additional content transformations are required.

Data Parsing

APIs dedicated to parsing tables and images with high accuracy and in different languages.

Advanced Retrieval

Enhanced algorithms such as hybrid search and reranking mechanisms.

Quality Assurance

Models for detecting and correcting hallucinations.

Security and Compliance

Components for encryption, data governance, role-based access controls, and PII processing.

Not only do you have to procure and on-board each of these systems or services, you need to integrate all of these within your RAG stack, and make sure they work harmoniously together, maintaining high uptime and low latency. Additionally, integrating them into your systems monitoring infrastructure and security processes is critical to identify vulnerabilities early and maintain overall system integrity.

Now, consider what happens when a bug is detected, latency rises above accepted thresholds, or response quality suddenly drops. You may find yourself working with multiple vendors, each with their own support staff and support Service Level Agreement (SLA), leaving you to be the coordinator between all these parties.

This is one of the areas where a more turn-key solution is extremely beneficial. Having the proverbial “one throat to choke” when something goes wrong can prevent endless headaches.

Team and Expertise

Another important challenge is building a team that can not only implement the initial RAG application, but also support and upgrade it over time,

making the necessary changes to support additional enterprise use cases.

If your RAG journey starts with a single use-case, it is to be expected that with success will come increased demand for additional use-cases. In fact, some large financial institutions have identified as many as 400-500 generative AI use-cases, and although this is not the same for every organization, we believe that most mature enterprises will be able to identify at least 30-50 use-cases of generative AI that provide significant value to their business operations in the first two years of using RAG technology.

RAG systems sit at the intersection of machine learning, software engineering, and domain-specific knowledge, necessitating teams with diverse competencies.

The primary challenge lies in assembling professionals who can bridge gaps between:

Machine Learning Engineering

Expertise in using embedding models correctly, using LLM inference and prompt engineering, implementing hybrid search architectures, optimizing retrieval pipelines, and specialized knowledge in hallucination detection and correction.

Data Engineering

Proficiency in building scalable ETL pipelines capable of handling unstructured data ingestion from diverse sources (PDFs, databases, etc).

DevOps/MLOps

Skills in containerization, continuous integration/deployment (CI/CD), and monitoring complex machine learning workflows.

Security/Compliance

skills in security, prompt injection prevention, PII redaction, data governance, data privacy and audit trails for generated content.

Not making this any easier—the knowledge about LLMs and RAG itself is relatively new, and continuously changes at a speed we’ve never seen before. Keeping up-to-date with all the best practices, and maintaining a deep understanding of the complexities involved can be quite challenging for most capable teams.

The challenges in assembling and maintaining highly skilled RAG teams stem from the rapid evolution of underlying technologies, the interdisciplinary nature of required skills. Each team member really needs to be an expert in their field, and the intense competition for AI talent, which is only expected to intensify further, makes this a real challenge.

To be successful with a high performance RAG solution in production, organizations must adopt an aggressive talent development strategy or choose to leverage turn-key RAG services for non-differentiating components while leveraging in-house domain expertise in the needs and requirements of the business.

Total Cost of Ownership

When migrating your RAG application from POC to production, it's helpful to consider the total cost of ownership required and plan your budget to make sure you would have enough budget support not only for a successful initial deployment, but also for additional use-cases of RAG and agentic RAG.

Here are the main components of a Total Cost of Ownership (TCO) for RAG:

Direct Costs

These are costs that you pay directly to a vendor for some service or component license that's included in your RAG stack.

- *Vendor management* for vector embeddings and LLMs. You will almost certainly need vendors for some components for the RAG pipeline; the question is if you have to manage many vendors,

payments, and cost optimization between different vendor components. Each additional vendor incurs security, legal, and IT overhead.

- *Retrieval pipeline operation* including expenses for running the retrieval pipeline, which encompasses the vector database, text retrieval (if employing hybrid search), and reranking. It is important to note that vector databases often exhibit non-linear cost scaling as increased data volumes require enhanced performance, such as low latency and high uptime.
- *Compute and Storage* costs for both staging and production environments often combining both CPU and GPU resources, to handle complex processing tasks.

Indirect and Ongoing Costs

Beyond the upfront expenditures, several indirect factors contribute to the overall TCO.

As your RAG stack grows, adding more data, expanding use cases, and increasing query volumes, your compute and storage needs will rise over time. Furthermore, ongoing support contracts with each vendor, regular system updates, and infrastructure monitoring can quickly add non-trivial additional costs.

Depending on your IT organization and its requirements, you may incur costs when integrating the RAG stack with existing enterprise software or third-party tools, implementing additional systems for data ingestion, testing, DevOps and monitoring as well as security and privacy controls.

Additional TCO Considerations

A thorough TCO evaluation should also factor in cybersecurity controls such as intrusion detection, regular audits, and any additional requirements your organization needs to comply with.

Now unfortunately, no system is completely resistant to downtime. Often at production scale a system is required to adhere to business continuity requirements and recovery solutions, adding even more to the costs.

In summary, successfully migrating your RAG application from POC to production requires a detailed and realistic assessment of both capital expenditures (CAPEX) and operational expenditures (OPEX). By accounting for direct vendor payments, compute and storage requirements, and a wide range of indirect costs, you can better prepare for the true costs of scaling your RAG solution.

We find that quite often initial cost estimates for DIY RAG deployments are notoriously unreliable, with actual production expenses often exceeding

projections by 3-5x.

So plan carefully, budget realistically, and consider long-term implications to achieve a successful, cost-effective production deployment.

Cost and TCO is another common reason companies prefer turnkey RAG solutions —where a single vendor provides all the functionality for RAG. In these cases, the vendor provides a significantly simplified cost structure, making the TCO much more predictable and manageable - both initially and over time.

RAG Evaluation

As we mentioned in “Response Quality and Reduced Hallucinations”, maintaining a healthy RAG pipeline with high quality responses and low hallucinations can be tricky as you scale up to enterprise scale. An important component that may help in addressing this challenge is a reliable RAG evaluation framework, which we will discuss in Chapter 6.

As the old adage goes: “you can’t fix what you can’t measure” - if you don’t have a reliable framework in place for measuring response quality and hallucination, your quality may degrade over time as you scale in production, and you may not realize it.

Continuously measuring your RAG pipeline often requires scalable and efficient implementation of retrieval metrics, generation metrics, as well as

overall end-to-end RAG response quality.

So we know the challenges of moving RAG to production, but not to despair. Many companies successfully navigate this transition. Next we are going to walk through some strategies to navigate this transition.

Successful Transition from POC to Production

Now that you know the risks and challenges, you are ready to plan your RAG production deployment.

Like with the deployment of any complex technology stack, careful planning can help in mitigating risks, and generative AI is no exception. In most cases, the POC already provided some initial hands-on experience, so you have a good set of questions to ask, and likely a good sense for what is important.

Summarize What you learned in the POC

Start with creating a report that summarizes all of the learnings from the POC. Here are some example questions and details that you might include in your report:

- Which components did you use in the POC: vector database, embedding model, LLM, re-ranker, etc
- How was data collected and ingested from source data stores?
- What was the prompt used for your RAG POC? How well did it work in terms of generating appropriate responses?
- Did response quality meet your expectations for the POC?
 - Latency. How was latency measured?
 - Response quality. How did you evaluate this?
- What unexpected issues did you uncover?
- What functionality did you not have in your POC and wanted to include, and why?

Once you write down this report, you are ready to define the goals for your actual production implementation.

Define Goals and Requirements

Before you get started with actual implementation, it really helps if you define the goals and requirements for your production deployment. In fact you might want to revisit the business goals to ensure that the POC's objectives align with your production goals.

Where applicable, use KPIs to define the requirements in a numeric form. You can fill in the results from the POC and define how much better you might want this to look in your production deployment.

Table 2-1 shows some of the KPIs and requirements that we've seen working with many Vectara customers—feel free to use this list, or adapt it to your needs. We filled in sample values for demonstrative purposes, but of course values from your POC or your goals for production may be different.

Table 2-1. Example list of RAG production system considerations

KPI / requirement	Definition	POC	Production
Query latency	Mean and median query response time (in seconds), measured over a set of 50 sample queries	Mean: 7.5 Median: 8.5	Mean: 4.5 Median: 4
Uptime and availability	The percentage of time the system is operational.	Not measured	Uptime >= 99.99%
Response quality	• Context Precision • Context Recall • Hallucination • Answer relevance	Not measured	CP >= 0.9 CR >= 0.8 Hallucination <= 0.05 AR >= 0.9
Data ingest	Data sources supported	Only local PDF files	File types: PDF, DOCX, PPTX,

KPI / requirement	Definition	POC	Production
	File types supported Requirement for refresh		HTML Sources: web pages, S3, snowflake DB, Notion Refreshes daily
Retrieval pipeline	What retrieval techniques are supported?	Vector search only	Vector search Hybrid search Relevance reranking Diversity reranking
Chunking	Which chunking strategies are supported	Fixed	Fixed Semantic
Data Security	Encryption on disk and in transit.	None	Must have
Access controls	Can response generation filter by	No	Must have

KPI / requirement	Definition	POC	Production
	user roles and permissions		
LLM selection	Which LLMs are supported for generation	OpenAI GPT-4o	OpenAI GPT-4o Anthropic Claude Llama 3.3 70B Deepseek-R1
Embedding model selection	Which embedding models are supported	Anything on Huggingface	Anything on HuggingFace OpenAI and Cohere

In addition to the items listed in [Table 2-1](#) above, there are other systems consideration to plan for in your production deployment:

Hardware

Consider which machines you need (both CPU and GPU machines), as well as memory capacity and networking requirements. Also

consider high availability requirements and staging environments which often require additional hardware.

Development environment and process

Where would code be hosted? Which CI/CD system will you use?

What unit testing or regression testing will you want to implement?

Data connectivity

Which enterprise systems does the RAG application need to connect to for data ingest, and how would credentials be provided? Consider implementing RBAC in your RAG to prevent data leakage against company policies.

Data security and governance

How does the system adhere with audit requirements, SOC-2 compliance, HIPAA compliance or GDPR (whatever is applicable in your organization)

Monitoring

how would you implement monitoring? Consider systems monitoring for uptime, latency monitoring, as well as user satisfaction (see Chapter 6 for RAG evaluation metrics).

Budget

What is the expected monthly budget allocated for the RAG application? How does performance degrade when you have a budget overrun?

Once the planning is complete, you can transition to implementation. This requires the traditional execution excellence skills in project management, agile development, and strong team coordination.

Successful technical implementation in your enterprise is highly dependent on R&D and IT practices, and is beyond the scope of this book.

So let's fast forward—your first production deployment of the first RAG use-case is two weeks out, and you are now ready to roll it organization wide. What comes next?

Ensuring Continued RAG success

First and foremost, you want to ensure a smooth and successful launch. This often requires training employees or customers with the new RAG application, making sure they are fully aware of all the capabilities and understand when to use it, and how to do so in the most effective way.

While users are using the RAG application, it's critical to pay careful attention to the metrics you have carefully built in. Not only user

satisfaction with query responses, but also latency, and systems performance.

You might, for example, see query volume peak in the first few days only to drop back to a much smaller volume of daily queries after 2-3 weeks. That likely indicates a problem somewhere—maybe the system is not providing useful responses to its users, and thus they revert back to their old ways of solving things. Maybe it's a latency issue and users don't want to wait.

It is not uncommon for some issues to arise in the first 2 weeks post deployment - things you didn't expect, or that did not come up during your pre-launch testing. So you want to make sure that you look at all the metrics, and react quickly to resolve any issues.

Having strong monitoring and observability capabilities as part of the implementation dramatically improves your chance of success. By looking at user queries and responses, by understanding latency metrics, and recording any issues that arise in day to day operations, you can quickly identify real application issues and move fast to remediate those.

Assuming the initial launch goes smoothly (outside of some issues you quickly move to remediate), there still remains a significant amount of work going forward. From menial tasks like systems maintenance, or compute upgrades as query volume and usage grows, to fixing systems uptime issues. You may need to upgrade components from time to time—for

example if you use a vector database it may need to be upgraded once you identify a security vulnerability.

More challenging is integrating new techniques into your RAG pipeline.

For example, let's imagine that a new embedding model is released that is shown to have a consistent 5% quality improvement across all your use-cases, wouldn't you want to adopt this? Of course you would. To do this you would have to implement this new model in your RAG pipeline (both at ingest and query time), test everything end-to-end, update any system dependencies and run an A/B test to prove that all works well and you see that 5% improvement.

This may be much easier said than done. For example, this new model may have a much higher latency. Or may require a different type of GPU machine which you might need to acquire or rent from a hyperscaler. And so on...

And this is just one example. It could be a new type of LLM, a better hybrid search algorithm, a new re-ranker, or an improved hallucination detection or correction component. In each upgrade to your RAG stack, make sure to follow a similar process as you did in the initial production deployment: plan, test, deploy and monitor.

Conclusion

Moving from a POC to production deployment of RAG in enterprise scale is not easy.

It requires a full understanding of all the requirements (security, governance, data privacy, systems operations) as well as maintaining a highly skilled team with a diverse expertise.

It's important to keep in mind that not only do you need to implement the first version of the RAG application, but also support ongoing maintenance, upgrades, and any issues that may arise. As the generative AI landscape evolves with new techniques to improve response quality and reduce hallucinations, better LLMs and embedding models, and more efficient components and hardware, keeping your system up-to-date may be challenging and requires considerable investment.

Most importantly, plan not only for continuous improvement but also for new use-cases your organization will want to implement to gain more benefit from your RAG stack.

Turn-key RAG platforms are quickly becoming a strong alternative to build-your-own. In this case, the vendor takes on this burden of quality implementation, upgrades, improvements, security, privacy and continuous monitoring, leaving the developers with the focus on the RAG application

itself - what data should it be based on, and where to integrate it into your business workflow.

In the next Chapter we will learn about Turn-key RAG platforms, what advantages they have over DIY systems and what are some of their limitations.

[OceanofPDF.com](https://oceanofpdf.com)